DALHOUSIE UNIVERSITY

Decision Trees

Recursive partitioning, impurity, and reading a fitted tree

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 A model you can read

What it is for Every classifier so far gives a label, and each explains it differently. kNN points at the neighbours that voted. LDA adds up weighted predictors and compares the sum with a threshold. An SVM with a radial kernel compares the case with its support vectors, and there is nothing else to read. A **decision tree** predicts by asking a short sequence of yes-or-no questions about the case, each in the case's own units, so the path to a prediction is its explanation. It is learned from data and evaluated with the usual tools, like every other model.

The data used throughout

Every worked example on growing a tree uses the same 28-row weather table. It has two categorical predictors (`Weather`, with levels rain, overcast and sunny, and `Humidity`, with levels high and normal), two numeric predictors (`Temp`, `Wind`) and a two-class target (`Decision`, Play or dontPlay). The class counts are 16 Play and 12 dontPlay. The 28 rows are the table L. Torgo used in his tree lectures at Dalhousie in 2021, in the style of Quinlan's weather data [3]. Section 11 then returns to the `Glass` data.

Table 1 lists the rows, and the first pair of listings types them in. Rows 15 to 28 repeat the `Weather`, `Humidity` and `Decision` of rows 1 to 14 with new `Temp` and `Wind`, so those three columns are typed once and repeated.

Table 1. The 28 rows of the weather data, in listing order (rows 15 to 28 on the right)

Weather	Temp	Humidity	Wind	Decision	Weather	Temp	Humidity	Wind	Decision
rain	26	high	15	dontPlay	rain	29	high	79	dontPlay
rain	35	normal	102	dontPlay	rain	26	normal	75	dontPlay
overcast	27	high	99	Play	overcast	30	high	108	Play
overcast	26	normal	97	Play	overcast	30	normal	16	Play
rain	12	high	120	Play	rain	33	high	96	Play
overcast	21	normal	74	Play	overcast	30	normal	13	Play
sunny	30	normal	89	dontPlay	sunny	32	normal	55	dontPlay
sunny	19	high	111	dontPlay	sunny	11	high	108	dontPlay
sunny	14	normal	81	Play	sunny	33	normal	103	Play
overcast	10	normal	70	Play	overcast	14	normal	32	Play
rain	11	normal	95	Play	rain	28	normal	44	Play
rain	15	high	94	Play	rain	21	high	84	Play
sunny	19	high	41	dontPlay	sunny	29	high	105	dontPlay
sunny	35	normal	38	dontPlay	sunny	15	normal	63	dontPlay

R

```r
# Rows 15 to 28 repeat the Weather, Humidity
# and Decision of rows 1 to 14.
w <- data.frame(
  Weather = rep(c("rain", "rain", "overcast",
    "overcast", "rain", "overcast", "sunny",
    "sunny", "sunny", "overcast", "rain",
    "rain", "sunny", "sunny"), 2),
  Temp = c(26, 35, 27, 26, 12, 21, 30, 19, 14,
    10, 11, 15, 19, 35, 29, 26, 30, 30, 33,
    30, 32, 11, 33, 14, 28, 21, 29, 15),
  Humidity = rep(c("high", "normal", "high",
    "normal", "high", "normal", "normal",
    "high", "normal", "normal", "normal",
    "high", "high", "normal"), 2),
  Wind = c(15, 102, 99, 97, 120, 74, 89, 111,
    81, 70, 95, 94, 41, 38, 79, 75, 108, 16,
    96, 13, 55, 108, 103, 32, 44, 84, 105, 63),
  Decision = rep(c("dontPlay", "dontPlay",
    "Play", "Play", "Play", "Play", "dontPlay",
    "dontPlay", "Play", "Play", "Play", "Play",
    "dontPlay", "dontPlay"), 2))
# Declared levels fix the order on every locale.
w$Decision <- factor(w$Decision,
                     levels = c("dontPlay", "Play"))
table(w$Weather, w$Decision)
#            dontPlay Play
#   overcast        0    8
#   rain            4    6
#   sunny           8    2
```

Python

```python
import numpy as np
import pandas as pd
# Rows 15 to 28 repeat the Weather, Humidity
# and Decision of rows 1 to 14.
w = pd.DataFrame({
    "Weather": ["rain", "rain", "overcast",
        "overcast", "rain", "overcast", "sunny",
        "sunny", "sunny", "overcast", "rain",
        "rain", "sunny", "sunny"] * 2,
    "Temp": [26, 35, 27, 26, 12, 21, 30, 19, 14,
        10, 11, 15, 19, 35, 29, 26, 30, 30, 33,
        30, 32, 11, 33, 14, 28, 21, 29, 15],
    "Humidity": ["high", "normal", "high",
        "normal", "high", "normal", "normal",
        "high", "normal", "normal", "normal",
        "high", "high", "normal"] * 2,
    "Wind": [15, 102, 99, 97, 120, 74, 89, 111,
        81, 70, 95, 94, 41, 38, 79, 75, 108, 16,
        96, 13, 55, 108, 103, 32, 44, 84, 105, 63],
    "Decision": ["dontPlay", "dontPlay", "Play",
        "Play", "Play", "Play", "dontPlay",
        "dontPlay", "Play", "Play", "Play", "Play",
        "dontPlay", "dontPlay"] * 2})
# The category order sets the crosstab columns.
w["Decision"] = pd.Categorical(w["Decision"],
    categories=["dontPlay", "Play"])
pd.crosstab(w["Weather"], w["Decision"])
# Decision  dontPlay  Play
# Weather
# overcast         0     8
# rain             4     6
# sunny            8     2
```

`pd.Categorical` is the counterpart of R's `factor` with declared levels, and naming the categories keeps the two cross-tabulations in the same order. The Python listings that follow reuse this `w` and these imports.

1.1 The tree this class grows

Figure 1 is the tree these notes build, fitted to the 28 rows on all four predictors with `rpart`, at most three questions deep and with at least three days in every leaf. Every tree has three kinds of node.

- the **root node** holds every case, here 16 Play and 12 dontPlay;
- an **internal node** asks a question about one predictor and sends each case to exactly one child;
- a **leaf** asks nothing and predicts the majority class of the cases that reached it.

The tree has six leaves and misclassifies 3 of the 28 days. The rest of these notes shows where each question comes from, how to read the tree for a new day, and why it stops where it does.

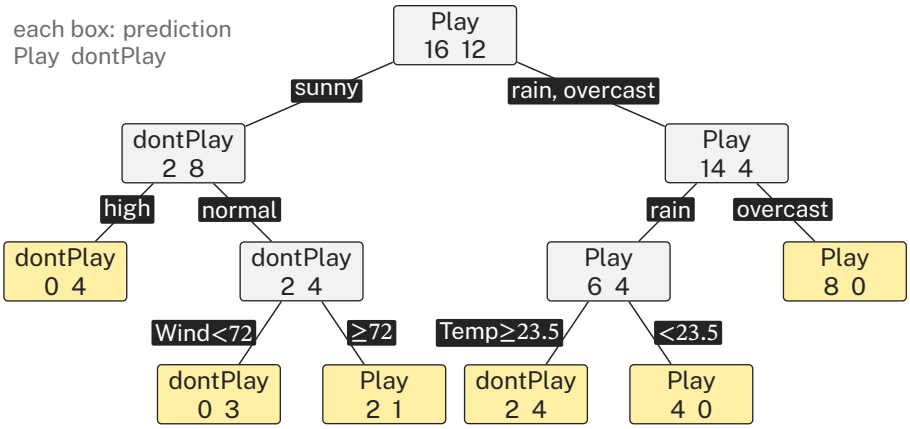


Figure 1. The running weather tree. Each box prints the predicted class above the Play and dontPlay counts of the days that reach it.

2 Growing a tree

2.1 The procedure

What it is for Recursive partitioning grows the tree. It runs the same steps on the cases that reach a node.

1. If the node should stop, because it is pure, holds too few cases to split, or sits as deep as allowed, make it a leaf that predicts the majority class of its cases.
2. Otherwise, for every predictor, list the questions its type allows, and score each one (Sections 4 to 6).
3. Keep the question with the best score.
4. Send the cases that answer yes one way and the rest the other way.
5. Repeat the procedure inside each of the two children.

The algorithm is **top-down**, because the whole sample is divided before any piece of it is. It is **recursive**, because step 5 is the same problem on a smaller set. And it is **greedy**, because step 3 takes the best question now and never revisits it. The tree that results is not guaranteed to be the best tree of its size, which is the price of a method that runs in seconds.

2.2 One split of the weather data

Worked example Split the 28 rows on `Weather`, sending the rain days one way and the overcast and sunny days the other, as Figure 2 shows.

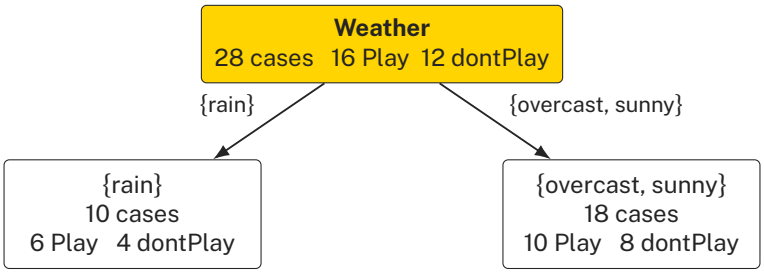


Figure 2. Splitting the weather data on rain versus overcast and sunny days

Every case goes to exactly one child and no case is lost, so the children's counts add back to the parent's, $10 + 18 = 28$ and $6 + 10 = 16$ Play. Each child is now a smaller copy of the same problem.

3 Three decisions the algorithm needs

Which question?

A test is good if it **splits** the cases of the sample into partitions that are "purer" than the parent node.

Which prediction?

A leaf predicts the value that best represents the cases in it.

When to stop?

Trees that are too large tend to **overfit** the training data and perform badly on new data, so the answer depends on how reliable an error estimate is.

A leaf predicts the majority class here, and the mean in a regression tree. Sections 4 and 5 answer which question to ask, and Section 10 shows why the impurity that chooses the question cannot say when to stop.

4 Which questions a tree can ask

What it is for Step 2 of the algorithm needs the list of candidate questions for each predictor, and the list depends on the predictor's type.

A category Send some levels one way and the rest the other way. For `Weather`, with levels rain, overcast and sunny, that gives three questions, "is it a rain day?", "is it overcast?" and "is it sunny?". A group of levels and the rest are the same question asked the other way round, so each grouping is counted once. A category with many levels has many groupings, 511 for ten levels. For a two-class target there is a shortcut. Order the levels by the share of one class among their cases, and try only the cuts along that order, because the best grouping is always one of them [1].

A number Sort the observed values and try a cut halfway between each pair of neighbours. The unsorted values of `Temp` are

```
26 35 27 26 12 21 30 19 14 10 11 15 19 35 29 26 30 30 33 30 32 11 33 14 28 21 29 15
```

which sort to

```
10 11 11 12 14 14 15 15 19 19 21 21 26 26 26 27 28 29 29 30 30 30 30 32 33 33 35 35
```

giving the questions *Temp* < 10.5, *Temp* < 11.5, *Temp* < 13, *Temp* < 14.5, and so on. There are 15 distinct values, so 14 candidate cuts, because repeated values leave no gap to cut in (Figure 3).

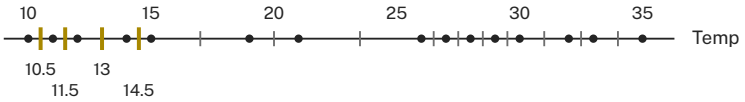


Figure 3. Candidate cuts along the sorted Temp values

No scaling needed Only the order of a numeric predictor's values decides which cases fall on each side of a cut. Standardizing, or any change of unit, keeps that order, so a tree needs no scaling, unlike kNN and the SVM.

5 How mixed is a box? Gini impurity

What it is for A good question leaves each side purer than the box it came from. The algorithm needs a number for "how mixed", so it can compare candidate questions.

The intuition Draw two cases at random from a box, putting the first back. The **Gini impurity** of the box is the chance that the two belong to different classes [1]. A pure box scores 0, because two draws never disagree. A 50–50 box of two classes scores 0.5, the most mixed two classes can be. The two draws agree when both land in the same class, and the chance that both land in class c is p_c^2, with $p_1, p_2, \ldots$ the shares of the classes in the box. One minus the chance of agreeing is

$$\text{Gini} = 1 - p_1^2 - p_2^2 - \cdots .$$

For the 28 days, 16 Play and 12 dontPlay, both draws are Play with chance $(16/28)^2 = 0.327$ and both dontPlay with chance $(12/28)^2 = 0.184$. They agree with chance 0.510, so the Gini impurity is $1 - 0.510 = 0.490$.

Scoring a question A question is worth the mixing it removes. Average the two children's Gini, weighting each by the share of the cases it holds, and subtract that average from the parent's Gini. For a box D of n cases that the question splits into D_L, with n_L cases, and D_R, with n_R,

$$\text{removed} = \text{Gini}(D) - \frac{n_L}{n}\,\text{Gini}(D_L) - \frac{n_R}{n}\,\text{Gini}(D_R).$$

Worked example Asking "is it a rain day?" sends 10 days one way, 6 Play and 4 dontPlay, and 18 days the other, 10 Play and 8 dontPlay.

Is it a rain day?	days	Play, dontPlay	Gini
yes	10	6, 4	$1 - 0.6^2 - 0.4^2 = 0.480$
no	18	10, 8	0.494
weighted average			$\frac{10}{28}(0.480) + \frac{18}{28}(0.494) = 0.489$
removed			$0.490 - 0.489 = 0.001$

Splitting the rain days off buys almost nothing, because both sides are about as mixed as the parent.

The other two questions Table 2 scores all three `Weather` questions. "Is it overcast?" gives a perfectly pure side, 8 days, all Play, yet loses to "is it sunny?". The pure side holds only 8 of the 28 days, so the remaining $20/28$ of the weight sits on 20 days, 8 Play and 12 dontPlay, whose Gini is $1 - 0.4^2 - 0.6^2 = 0.480$, barely below the parent's 0.490. That leaves $\frac{8}{28}(0) + \frac{20}{28}(0.480) = 0.343$. The sunny question sends 10 days (2 Play, 8 dontPlay, Gini $1 - 0.2^2 - 0.8^2 = 0.320$) one way and 18 days (14 Play, 4 dontPlay, Gini $1 - (14/18)^2 - (4/18)^2 = 0.346$) the other, so $\frac{10}{28}(0.320) + \frac{18}{28}(0.346) = 0.337$ is left. That is lower, so sunny removes more mixing in total. A small pure side counts only for the cases it holds.

Table 2. Gini after each `Weather` question and the mixing it removes

	answer yes		answer no		
Question	days (Play, dontPlay)	Gini	days (Play, dontPlay)	Gini	Gini after, removed
Is it rain?	10 (6, 4)	0.480	18 (10, 8)	0.494	0.489, 0.001
Is it overcast?	8 (8, 0)	0	20 (8, 12)	0.480	0.343, 0.147
Is it sunny?	10 (2, 8)	0.320	18 (14, 4)	0.346	**0.337, 0.153**

A number cut, worked A cut on a number is scored the same way. The question `Temp` < 14.5 sends the six coldest days left, at 10, 11, 11, 12, 14 and 14 degrees. Five of them are Play, so that side scores $1 - (5/6)^2 - (1/6)^2 = 0.278$. The other 22 days split 11 to 11 and score 0.500. The weighted average is $\frac{6}{28}(0.278) + \frac{22}{28}(0.500) = 0.452$, so the cut removes $0.490 - 0.452 = 0.037$. Figure 4 repeats this for all 14 cuts. The scores do not rise smoothly toward one good threshold. They jump from cut to cut, because moving the cut by one value moves one or two days across it. The best cut, `Temp` < 34, removes 0.050 only by cutting off the two days at 35 degrees, both dontPlay, which is the small pure side of the overcast question again. A rule that every side must hold at least three days, used later in these notes, rules that cut out.

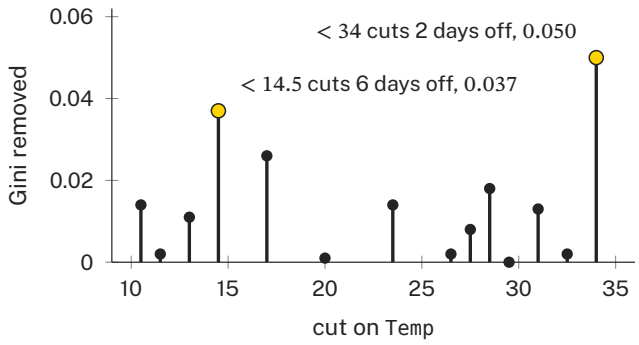


Figure 4. The Gini each of the 14 `Temp` cuts removes from the 28 days. The gold marks are the cut worked above and the best cut.

The same arithmetic runs in both languages. `gini` measures a box, and `gain` the mixing a question removes. `gain` takes the impurity as its first argument, so the entropy of Section 7 reuses it.

R

```r
gini <- function(y) {
  p <- prop.table(table(y))
  1 - sum(p^2)
}
gain <- function(impurity, y, keep) {
  n <- length(y)   # mixing removed by the split
  left <- sum(keep) / n * impurity(y[keep])
  right <- sum(!keep) / n * impurity(y[!keep])
  impurity(y) - left - right
}
y <- w$Decision; wx <- w$Weather
gini(y)                          # 0.4897959
gain(gini, y, wx == "rain")      # 0.0009070295
gain(gini, y, wx == "overcast")  # 0.1469388
gain(gini, y, wx == "sunny")     # 0.153288
```

Python

```python
def gini(y):
    p = y.value_counts(normalize=True).to_numpy()
    return 1 - (p ** 2).sum()

def gain(impurity, y, keep):  # mixing removed
    n = len(y)
    left = keep.sum() / n * impurity(y[keep])
    right = (~keep).sum() / n * impurity(y[~keep])
    return impurity(y) - left - right

y, wx = w["Decision"], w["Weather"]
gini(y)                          # 0.4897959
gain(gini, y, wx == "rain")      # 0.0009070
gain(gini, y, wx == "overcast")  # 0.1469388
gain(gini, y, wx == "sunny")     # 0.1532880
```

6 The search, at the root and below it

What it is for Step 2 of the algorithm scores every question on every predictor, and step 5 runs the same search again inside each child. This section does both on the weather data.

Worked example At the root there are 44 questions, 3 on `Weather`, 14 on `Temp`, 1 on `Humidity` and 26 on `Wind`. The left half of Table 3 keeps the best question of each predictor. `Weather = sunny` removes 0.153, three times as much as the best number cut, `Temp < 34`, so it becomes the root.

The 18 days that are not sunny now form their own box, with 14 Play and 4 dontPlay. The search runs again on those 18 days alone, and the right half of the table shows the result. `Weather` now has only two levels left, so one question, and `Weather = overcast` wins with 0.079. That is the second question of Figure 1. Nothing about the root is reused except the days it sent here. The same step on the 10 sunny days chooses `Humidity`.

Table 3. The best question of each predictor and the Gini it removes, at the root and inside the 18 days that are not sunny

	all 28 days			the 18 not sunny		
Predictor	tried	best question	removed	tried	best question	removed
Weather	3	= sunny	**0.153**	1	= overcast	**0.079**
Temp	14	< 34	0.050	12	< 34	0.071
Humidity	1	= high	0.008	1	= high	0.001
Wind	26	< 80	0.021	17	< 81.5	0.025

R

```r
questions <- function(x) {  # every yes-or-no question
  if (is.numeric(x)) {      # midpoints of values
    v <- sort(unique(x))
    c <- (head(v, -1) + tail(v, -1)) / 2
    return(setNames(lapply(c, \(t) x < t), paste("<", c)
  ↪ ))
  }
  lv <- sort(unique(x))     # one level vs the rest
  if (length(lv) == 2) lv <- lv[1]
  setNames(lapply(lv, \(v) x == v), paste("=", lv))
}
best <- function(d) for (col in names(d)[1:4]) {
  s <- sapply(questions(d[[col]]),  # every question
              function(k) gain(gini, d$Decision, k))
  cat(col, length(s), names(which.max(s)),
      round(max(s), 3), "\n")      # and the best
}
best(w)                            # all 28 days
best(w[w$Weather != "sunny", ])    # the 18 others
# Weather 3 = sunny 0.153       the 28 days
# Temp 14 < 34 0.05
# Humidity 1 = high 0.008
# Wind 26 < 80 0.021
# Weather 1 = overcast 0.079    the 18 not sunny
# Temp 12 < 34 0.071
# Humidity 1 = high 0.001
# Wind 17 < 81.5 0.025
```

Python

```python
def questions(x):  # every yes-or-no question on x
    if pd.api.types.is_numeric_dtype(x):
        v = np.unique(x)   # midpoints of sorted values
        cuts = (v[:-1] + v[1:]) / 2
        return {f"< {c:g}": x < c for c in cuts}
    lv = sorted(x.unique())  # one level vs the rest
    lv = lv[:1] if len(lv) == 2 else lv
    return {"= " + v: x == v for v in lv}

def best(d):            # best question per predictor
    for col in d.columns[:4]:
        s = {q: gain(gini, d["Decision"], k)
             for q, k in questions(d[col]).items()}
        q = max(s, key=s.get)
        print(col, len(s), q, round(s[q], 3))

best(w)                             # all 28 days
best(w[w["Weather"] != "sunny"])    # the 18 others
```

The scan of Figure 4 is `questions` and `gain` applied to `Temp` alone, printing each cut, the days below it, the Play days among them, and the Gini it removes.

R

```r
for (q in names(questions(w$Temp))) {
  k <- questions(w$Temp)[[q]]
  cat(q, sum(k), sum(y[k] == "Play"),
      round(gain(gini, y, k), 3), "\n")
}
# cut, days below, Play below, Gini removed
# < 10.5 1 1 0.014
# < 11.5 3 2 0.002
# < 13 4 3 0.011
# < 14.5 6 5 0.037
# < 17 8 6 0.026
# < 20 10 6 0.001
# < 23.5 12 8 0.014
# < 26.5 15 9 0.002
# < 27.5 16 10 0.008
# < 28.5 17 11 0.018
# < 29.5 19 11 0
# < 31 23 14 0.013
# < 32.5 24 14 0.002
# < 34 26 16 0.05
```

Python

```python
for c, k in questions(w["Temp"]).items():
    print(c, k.sum(), (y[k] == "Play").sum(),
          round(gain(gini, y, k), 3))
# cut, days below, Play below, Gini removed
# < 10.5 1 1 0.014
# < 11.5 3 2 0.002
# < 13 4 3 0.011
# < 14.5 6 5 0.037
# < 17 8 6 0.026
# < 20 10 6 0.001
# < 23.5 12 8 0.014
# < 26.5 15 9 0.002
# < 27.5 16 10 0.008
# < 28.5 17 11 0.018
# < 29.5 19 11 0.0
# < 31 23 14 0.013
# < 32.5 24 14 0.002
# < 34 26 16 0.05
```

6.1 The whole tree, node by node

Worked example The same search runs at every node, on the days that reached it, until a node stops. The running tree of Figure 1 adds two settings. A question is allowed only if each side keeps at least three days, and no path asks more than three questions. Table 4 follows the growth from the root to the six leaves.

Three leaves stop because they are pure, and nothing is left to separate. The other three stop because they sit three questions deep. Two of those could not improve anyway. The six rain days at 23.5

Table 4. Every node of the running tree, the best allowed question and the Gini it removes, and why each leaf stops

Node	days (Play, dontPlay)	Gini	best question	removed	then
all days	28 (16, 12)	0.490	Weather = sunny	0.153	split
sunny	10 (2, 8)	0.320	Humidity = high*	0.053	split
high	4 (0, 4)	0			stop, pure
normal	6 (2, 4)	0.444	Wind < 72	0.222	split
Wind < 72	3 (0, 3)	0			stop, pure
Wind ≥ 72	3 (2, 1)	0.444			stop, depth 3
not sunny	18 (14, 4)	0.346	Weather = overcast	0.079	split
overcast	8 (8, 0)	0			stop, pure
rain	10 (6, 4)	0.480	Temp < 23.5	0.213	split
Temp < 23.5	4 (4, 0)	0			stop, pure
Temp ≥ 23.5	6 (2, 4)	0.444			stop, depth 3

degrees or more, 2 Play and 4 dontPlay, have no allowed question that removes any Gini, since every split into two groups of three leaves the same mix of classes.

The sunny node needs a warning. `Humidity = high` and `Wind < 72` cut off different days, yet each leaves four dontPlay days on one side and 2 Play and 4 dontPlay on the other, so both remove exactly 0.053. `rpart` keeps the predictor named first in the formula, `Humidity`, and scikit-learn, whose choice depends on `random_state`, picks it too. Name `Wind` before `Humidity` in the `rpart` formula and the tie breaks the other way. The sunny branch then asks `Wind < 72` and `Wind ≥ 104`, the tree still has six leaves and three training errors, and `Humidity` appears nowhere in it. The order of the columns decided whether humidity looks relevant. `top` below finds the best allowed question at a node, and the first best wins.

R

```r
top <- function(d, m = 3) {  # m+ days a side
  s <- unlist(lapply(names(d)[1:4], function(col) {
    q <- questions(d[[col]])
    q <- q[sapply(q, function(k)
      sum(k) >= m && sum(!k) >= m)]
    if (length(q) == 0) return(NULL)
    g <- sapply(q, function(k)
      gain(gini, d$Decision, k))
    setNames(g, paste(col, names(q)))
  }))
  round(s[which.max(s)], 3)   # first best wins
}
sun <- wx == "sunny"; rain <- wx == "rain"
normal <- w$Humidity == "normal"
top(w)                  # Weather = sunny 0.153
top(w[sun, ])           # Humidity = high 0.053
top(w[sun & normal, ]) # Wind < 72 0.222
top(w[!sun, ])          # Weather = overcast 0.079
top(w[rain, ])          # Temp < 23.5 0.213
```

Python

```python
def top(d, m=3):   # best question, m+ days a side
    s = {(col, q): gain(gini, d["Decision"], k)
         for col in d.columns[:4]
         for q, k in questions(d[col]).items()
         if m <= k.sum() <= len(d) - m}
    (col, q), g = max(s.items(), key=lambda t: t[1])
    return col, q, round(g, 3)

sun, rain = wx == "sunny", wx == "rain"
normal = w["Humidity"] == "normal"
top(w)               # ('Weather', '= sunny', 0.153)
top(w[sun])          # ('Humidity', '= high', 0.053)
top(w[sun & normal])     # ('Wind', '< 72', 0.222)
top(w[~sun])         # ('Weather', '= overcast', 0.079)
top(w[rain])         # ('Temp', '< 23.5', 0.213)
```

7 Entropy, the second measure of mixing

What it is for Some libraries and older texts score questions with entropy instead of Gini. It plays the same role and is computed the same way, so the only question is whether it changes the tree.

The rule The **entropy** of a box measures, in bits, how uncertain a case's class is. A 50–50 box of two classes is 1 bit, the answer to one fair yes-or-no question, and a pure box is 0 bits. With p_c the share of class c,

$$H = -\sum_c p_c \log_2 p_c,$$

with $0 \log_2 0$ counted as 0. A question is scored by the entropy it removes, the parent's H minus the weighted average of the children's, exactly as for Gini. This drop is called the **information gain**.

Worked example The 28 days have $H = -\frac{16}{28}\log_2\frac{16}{28} - \frac{12}{28}\log_2\frac{12}{28} = 0.985$ bits. The overcast question leaves a pure side ($H = 0$) and 20 days, 8 Play and 12 dontPlay, with $H = -0.4\log_2 0.4 - 0.6\log_2 0.6 = 0.529 + 0.442 = 0.971$, removing $0.985 - \frac{20}{28}(0.971) = 0.292$ bits. The sunny question leaves 0.722 and 0.764 bits and removes 0.236. Entropy therefore picks overcast, where Gini picked sunny. Figure 5 shows why. Doubling Gini gives it the same maximum as entropy, 1 at 50–50, and the two curves then agree near the middle and part toward the ends. The overcast question's mixed side, 8 to 12, sits near the middle, where entropy gives 0.971 and doubled Gini 0.960. The sunny question's sides are lopsided, 2 to 8 and 14 to 4, where entropy gives 0.722 and 0.764 against 0.640 and 0.691. Entropy charges lopsided boxes more, so it marks the sunny question down, while the pure side of the overcast question costs nothing under either.

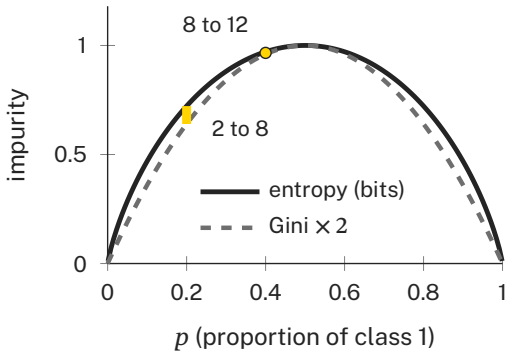


Figure 5. Entropy (solid) and Gini impurity doubled (dashed) against the share of one class. The gold bar is the gap at the sunny side (2 to 8), and the gold dot marks the overcast question's mixed side (8 to 12), where the curves almost meet.

On most data the two measures choose the same question. Here the gap between the two Weather questions is small, 0.006 in Gini, so the measure tips it. `rpart` with `parms = list(split = "information")` and scikit-learn with `criterion="entropy"` both put overcast at the root. Report which measure grew your tree.

R

```r
entropy <- function(y) {
  p <- prop.table(table(y))
  p <- p[p > 0]        # 0 log 0 counts as 0
  -sum(p * log2(p))
}
entropy(y)                          # 0.9852281
gain(entropy, y, wx == "overcast")  # 0.291692
gain(entropy, y, wx == "sunny")     # 0.2361223
round(c(entropy(y[wx != "overcast"]),  # the sides
  entropy(y[wx == "sunny"]),
  entropy(y[wx != "sunny"])), 3)
# 0.971 0.722 0.764
round(2 * c(gini(y[wx != "overcast"]),  # Gini x 2
  gini(y[wx == "sunny"]), gini(y[wx != "sunny"])), 3)
# 0.96 0.64 0.691
t <- rpart(Decision ~ ., data = w,  # entropy root
  method = "class", parms = list(split =
  "information"), control = list(maxdepth = 1))
labels(t, minlength = 0)[2]  # "Weather=rain,sunny"
```

Python

```python
def entropy(y):
    p = y.value_counts(normalize=True).to_numpy()
    p = p[p > 0]            # 0 log 0 counts as 0
    return -(p * np.log2(p)).sum()

entropy(y)                          # 0.9852281
gain(entropy, y, wx == "overcast") # 0.2916920
gain(entropy, y, wx == "sunny")    # 0.2361223
[round(entropy(y[k]), 3) for k in   # the sides
 (wx != "overcast", wx == "sunny", wx != "sunny")]
# [0.971, 0.722, 0.764]
[round(2 * gini(y[k]), 3) for k in  # Gini x 2
 (wx != "overcast", wx == "sunny", wx != "sunny")]
# [0.96, 0.64, 0.691]
t = tree.DecisionTreeClassifier(   # entropy root
    criterion="entropy", max_depth=1
).fit(X4, y)
X4.columns[t.tree_.feature[0]]     # 'Weather_overcast'
```

8 Reading a fitted tree

What it is for A fitted tree is used by sending a new case down it. The path it takes is the explanation of its prediction, and the leaf it reaches says how sure that prediction is.

Worked example A new day is rain, `Temp` 25, normal humidity and `Wind` 40. In Figure 1 it is not sunny, so it goes right. It is a rain day, so it goes left at the second question. Its `Temp` is at least 23.5, so it lands in the leaf that holds 2 Play and 4 dontPlay days. The tree predicts **dontPlay**, and the class shares in that leaf, $4/6 = 0.67$ dontPlay and $2/6 = 0.33$ Play, are what `predict(..., type = "prob")` and `predict_proba` return. `Humidity` and `Wind` are never asked on this path, so they cannot change this day's prediction.

Three rules apply when reading any fitted tree.

1. **A leaf's label is a majority vote.** The leaf above is right about 4 of its 6 training days. Its counts are the evidence behind its label.
2. **A small leaf rests on little evidence.** The `Wind` ≥ 72 leaf predicts Play from 3 days, 2 of them Play.
3. **A predictor the tree never asks about may still matter.** It was never the best question at any node, often because a predictor already used carries the same information, and sometimes only because it lost a tie. Section 6.1 shows a tree in which `Humidity` disappears when the column order changes.

The printouts Both listings fit the tree with Gini, at most three questions deep and at least three days in a leaf. `rpart` takes `Weather` as a category and searches groupings of its levels. scikit-learn needs numbers, so `get_dummies` turns each level into a 0 or 1 column, and `Weather_sunny <= 0.5` reads as "not sunny".

R

```r
library(rpart)
fit <- rpart(Decision ~ Weather + Temp +
    Humidity + Wind, data = w, method = "class",
  control = rpart.control(cp = 0, maxdepth = 3,
    minsplit = 6, minbucket = 3))
print(fit, digits = 3)
# node), split, n, loss, yval, (yprob)
#  1) root 28 12 Play (0.429 0.571)
#    2) Weather=sunny 10 2 dontPlay (0.800 0.200)
#      4) Humidity=high 4 0 dontPlay (1.000 0.000) *
#      5) Humidity=normal 6 2 dontPlay (0.667 0.333)
#       10) Wind< 72 3 0 dontPlay (1.000 0.000) *
#       11) Wind>=72 3 1 Play (0.333 0.667) *
#    3) Weather=overcast,rain 18 4 Play (0.222 0.778)
#      6) Weather=rain 10 4 Play (0.400 0.600)
#       12) Temp>=23.5 6 2 dontPlay (0.667 0.333) *
#       13) Temp< 23.5 4 0 Play (0.000 1.000) *
#      7) Weather=overcast 8 0 Play (0.000 1.000) *
```

Python

```python
X4 = pd.get_dummies(w.drop(columns="Decision"),
    dtype=int)              # 0/1 per level
fit = tree.DecisionTreeClassifier(
    criterion="gini", max_depth=3,
    min_samples_split=6, min_samples_leaf=3,
    random_state=0).fit(X4, w["Decision"])
print(tree.export_text(fit, spacing=2,
    decimals=1, show_weights=True,
    feature_names=list(X4.columns)))
# |-- Weather_sunny <= 0.5
# |  |-- Weather_rain <= 0.5
# |  |  |-- weights: [8.0, 0.0] class: Play
# |  |-- Weather_rain >  0.5
# |  |  |-- Temp <= 23.5
# |  |  |  |-- weights: [4.0, 0.0] class: Play
# |  |  |-- Temp >  23.5
# |  |  |  |-- weights: [2.0, 4.0] class: dontPlay
# |-- Weather_sunny >  0.5
# |  |-- Humidity_high <= 0.5
# |  |  |-- Wind <= 72.0
# |  |  |  |-- weights: [0.0, 3.0] class: dontPlay
# |  |  |-- Wind >  72.0
# |  |  |  |-- weights: [2.0, 1.0] class: Play
# |  |-- Humidity_high >  0.5
# |  |  |-- weights: [0.0, 4.0] class: dontPlay
```

In R's printout each line is one node, numbered so that node k has children $2k$ and $2k+1$, followed by its question, its number of days, the days it misclassifies, its prediction and its class shares in the order dontPlay, Play. A star marks a leaf. In scikit-learn's printout each line is one branch, and `weights` lists the leaf's counts in the order of `classes_`, Play first. The two trees have the same six leaves. The prediction for the new day runs the same way in both.

R

```r
day <- data.frame(Weather = "rain", Temp = 25,
                  Humidity = "normal", Wind = 40)
predict(fit, day, type = "prob")
#    dontPlay      Play
# 1 0.6666667 0.3333333
predict(fit, day, type = "class")   # dontPlay
```

Python

```python
day = pd.DataFrame({"Weather": ["rain"],
    "Temp": [25], "Humidity": ["normal"],
    "Wind": [40]})
day = pd.get_dummies(day, dtype=int).reindex(
    columns=X4.columns, fill_value=0)
print(fit.classes_)         # ['Play' 'dontPlay']
print(fit.predict_proba(day))  # [[0.333 0.667]]
print(fit.predict(day))     # ['dontPlay']
```

9 One object, two pictures

What it is for A tree and the regions it cuts the predictor space into are the same model drawn two ways. Every internal node is one cut, every leaf one region, and the prediction is constant inside each region. With two numeric predictors the regions are rectangles, and the whole model fits on one pair of axes.

Worked example Figure 6 grows a tree on the 28 rows using `Temp` and `Wind`, at most two questions deep and with at least three cases in a leaf.

The pair of pictures shows three properties of every tree.

1. **Every cut is parallel to an axis.** A tree draws a diagonal boundary only as a staircase of many cuts, where LDA draws one straight line and the SVM a smooth curve.

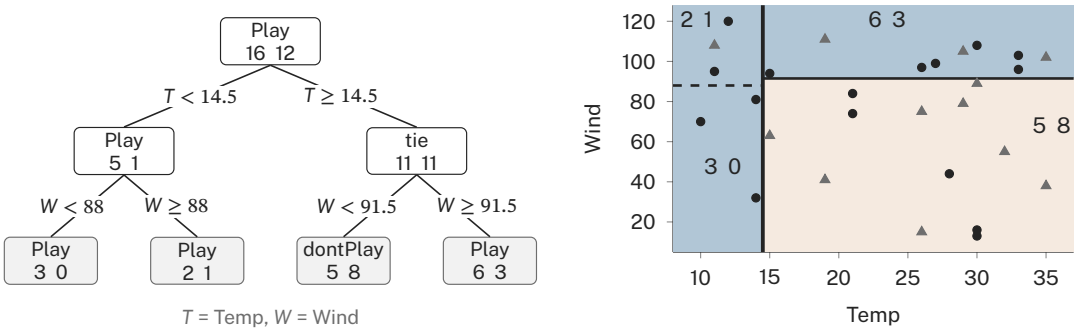


Figure 6. A tree and its partition of the Temp and Wind plane. Filled dots are Play and triangles are dontPlay; blue regions predict Play and rust regions dontPlay. The warm node is an 11–11 tie, which `rpart` prints as dontPlay.

2. **A cut below the root applies only inside its own rectangle.** The two `Wind` cuts sit at different heights, 88 on the cold side and 91.5 on the warm side, because each was chosen using only the cases in its own branch.
3. **A split can change no prediction.** Both children of the cold branch predict Play, so the `Wind` < 88 cut only sharpens the class shares. Pruning, coming next, removes splits like this one.

10 When to stop growing

10.1 Training accuracy always rises

What it is for Impurity decides which question to ask, but not when to stop asking. A split can only leave the boxes as mixed as before or purer, so training accuracy never falls as the tree grows. Stopping needs separate evidence.

Worked example Figure 7 grows trees of increasing maximum depth on the `Glass` data. Each depth is scored with balanced accuracy on the 214 rows it was fitted to, and on each of five held-out folds by a tree refitted to the other four. The training score climbs from 0.31 at depth 1 to 0.99 at depth 10. The held-out folds climb to about 0.61 by depth 4 and then wander between 0.57 and 0.62. These are the folds the Python listing draws. In R, `rpart`'s training curve also climbs to 0.99, and on R's own folds the held-out curve levels off between 0.62 and 0.66. The widening gap is overfitting. The extra depth fits quirks of the training rows that the held-out rows do not share.

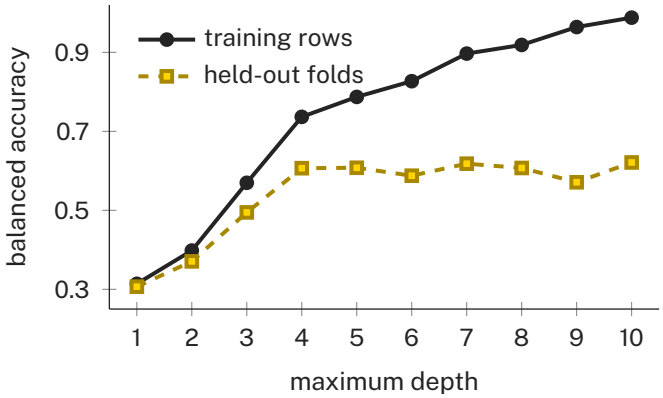


Figure 7. Balanced accuracy of a tree on `Glass` against its maximum depth, on the training rows and on the held-out folds

10.2 An unstopped tree memorizes

Worked example Take the 28 weather rows and the two numeric predictors `Temp` and `Wind`. No two of the 28 rows share a (`Temp`, `Wind`) pair, so whenever a node still holds two cases of different classes, some cut separates them. Grown until no such node is left, the tree has 12 leaves, reaches depth 7, and misclassifies none of the 28 days. Four of its twelve leaves hold a single day (Figure 8).

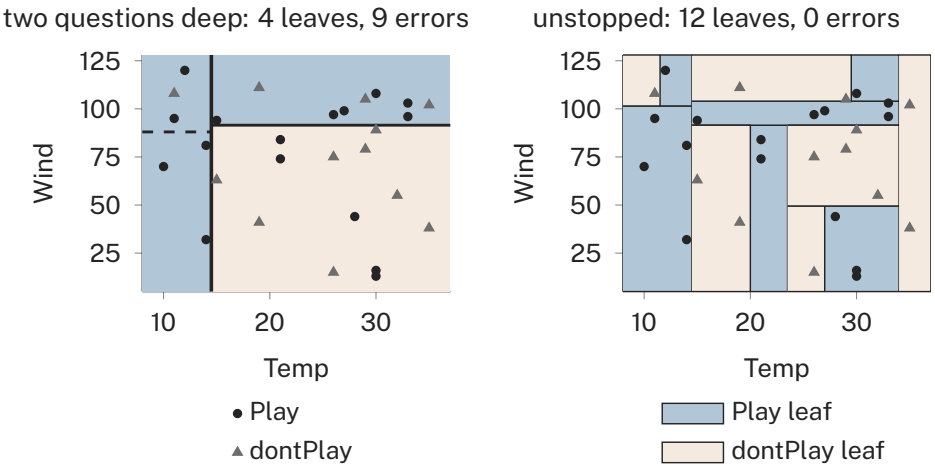


Figure 8. The partitions of the stopped and unstopped trees

The right-hand partition describes these 28 days and nothing beyond them. Its zero training error follows from the construction, as the perfect training score of 1-NN does, and it says nothing about a 29th day.

10.3 Two ways to stop

Further down the tree each question is chosen on fewer and fewer cases, so it is less reliable. The deepest split of the unstopped tree was chosen using four days. The simplest stopping rule refuses to make a leaf with fewer than m days. Table 5 grows the `Temp` and `Wind` tree three times.

Table 5. Leaves and training errors of the `Temp` and `Wind` tree as the smallest leaf size grows (scikit-learn; `rpart` at `cp = 0` prints 12, 7 and 3 leaves, and at `cp = -1` 12, 8 and 4)

Smallest leaf m	Leaves	Training errors
1	12	0
3	8	6
5	4	9

R

```r
for (m in c(1, 3, 5)) {      # smallest leaf allowed
  t <- rpart(Decision ~ Temp + Wind, data = w,
    method = "class", control = rpart.control(
      cp = 0, minsplit = 2 * m, minbucket = m))
  cat(m, sum(t$frame$var == "<leaf>"),
    sum(predict(t, w, type = "class") != w$Decision),
    "\n")
}
# 1 12 0
# 3 7 6
# 5 3 9
```

Python

```python
for m in (1, 3, 5):          # smallest leaf allowed
    t = tree.DecisionTreeClassifier(
        min_samples_leaf=m, random_state=15
    ).fit(X, w["Decision"])
    print(m, t.get_n_leaves(),
          (t.predict(X) != w["Decision"]).sum())
# 1 12 0
# 3 8 6
# 5 4 9
```

A larger m buys leaves that rest on several days, and pays for them in training errors. The training errors always prefer $m = 1$, so they cannot choose m. There are two responses.

- **Stop early.** Limit the depth, or refuse to make a leaf with too few cases. In `rpart` these are `maxdepth`, `minsplit` and `minbucket`, and in scikit-learn `max_depth`, `min_samples_split` and `min_samples_leaf`. This is also called pre-pruning.
- **Grow, then cut back.** Grow a large tree and remove the branches that cross-validation does not support. This is post-pruning, the standard procedure, and pruning, coming next, builds it step by step. It is preferred because a question that looks worthless can enable a valuable one beneath it, and a rule that stops early never finds out.

11 Choosing the depth and comparing with earlier models

What it is for A depth limit is a setting, like K in kNN or C and γ in the SVM, so it is chosen the way they are. Inside each of the five outer training sets of `Glass`, five inner folds choose a maximum depth from 1 to 10, and the chosen tree is scored once on its outer fold. The folds are drawn as they were for the SVM. Python's `StratifiedKFold` with `random_state=3141` gives the outer folds and, applied to each outer training set, its inner folds. R draws its own with `createFolds` after `set.seed(3141)`.

R

```r
data(Glass, package = "mlbench")
library(caret)
set.seed(3141)          # R's own folds
outer <- createFolds(Glass$Type, k = 5,
                     list = FALSE)
bacc <- function(y, p)    # balanced accuracy
  mean(tapply(p == y, y, mean))
grow <- function(d, depth) rpart(Type ~ ., d,
  control = rpart.control(cp = 0, minsplit = 2,
    minbucket = 1, maxdepth = depth, xval = 0))
gini(Glass$Type)   # 0.7367, six classes
```

Python

```python
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import (
    GridSearchCV, StratifiedKFold)

url = ("https://archive.ics.uci.edu/ml/"
       "machine-learning-databases/glass/glass.data")
glass = pd.read_csv(url, header=None, index_col=0)
X = glass.iloc[:, :-1].to_numpy()
y = glass.iloc[:, -1].astype(str).to_numpy()
folds = StratifiedKFold(5, shuffle=True,
                        random_state=3141)
grid = {"max_depth": list(range(1, 11))}
gini(pd.Series(y))   # 0.7367, six classes
```

R

```r
outer_score <- function(k) {
  d <- Glass[outer != k, ]
  inner <- createFolds(d$Type, 5, FALSE)
  cv <- function(depth) mean(sapply(1:5,
    function(j) {
      m <- grow(d[inner != j, ], depth)
      p <- predict(m, d[inner == j, ], "class")
      bacc(d$Type[inner == j], p)
    }))          # choose the depth INSIDE
  m <- grow(d, which.max(sapply(1:10, cv)))
  test <- Glass[outer == k, ]
  bacc(test$Type, predict(m, test, "class"))
}               # score OUTSIDE
round(sapply(1:5, outer_score), 3)
# 0.631 0.636 0.821 0.497 0.673
```

Python

```python
outer = []
for tr, te in folds.split(X, y):
    gs = GridSearchCV(                          # INSIDE
        tree.DecisionTreeClassifier(random_state=0),
        grid, cv=folds, scoring="balanced_accuracy")
    gs.fit(X[tr], y[tr])
    outer.append(balanced_accuracy_score(     # OUTSIDE
        y[te], gs.predict(X[te])))
print(np.round(outer, 3))
# [0.775 0.554 0.683 0.558 0.571]
```

The two languages print different scores for two reasons. R and Python draw different random numbers, so R's folds are not Python's. And `rpart` breaks ties between equally good questions differently and drops a split that changes no prediction even at `cp = 0`, so it grows slightly different trees. R's outer mean is 0.652, against Python's 0.628.

The comparison Table 6 adds Python's tree to the four earlier models, all on the same outer folds, the ones the Python listing draws. The tree's mean, 0.628, sits 0.002 below kNN's (0.630) and 0.015 above the SVM's (0.613). The corrected resampled test, with the test-to-train ratio $1/4$ of five folds, gives a standard error of 0.057 for the tree-minus-kNN gap. With $t = 2.78$, the value that leaves 2.5% in each tail of a t distribution with 4 degrees of freedom, the 95% interval is $-0.002 \pm 2.78 \times 0.057$, from -0.16 to 0.16 ($p = 0.98$). Against the SVM the standard error is 0.112 and the interval runs from -0.30 to 0.33 ($p = 0.90$). Both intervals cover zero by a wide margin, so correcting for having made two comparisons changes nothing. Five folds cannot tell these models apart. A gap of 0.16 in either direction is as consistent with them as no gap at all. The readable model shows no measurable cost on `Glass`, and these five folds cannot rule out a cost of that size.

Table 6. Outer-fold balanced accuracy on `Glass`, with the depth each outer fold chose

Outer fold	1	2	3	4	5	mean
Baseline	0.167	0.167	0.167	0.167	0.167	0.167
LDA	0.574	0.613	0.452	0.608	0.481	0.546
kNN	0.638	0.583	0.685	0.581	0.663	0.630
SVM	0.553	0.421	0.679	0.648	0.766	0.613
Tree	0.775	0.554	0.683	0.558	0.571	0.628
Tree depth chosen	10	5	10	6	10	

The inner search chose depth 10, the largest allowed, on three of the five folds, yet Figure 7 is flat after depth 4. The search picks the highest inner score however small its lead, so it drifts to deep trees that buy nothing reliable. Pruning, coming next, answers with the one-standard-error rule, which keeps the simplest tree that does as well.

12 Instability

What it is for A tree's questions are chosen greedily, each on the cases its parent sent it, so a small change at the top changes everything below. This is the cost of a model you can read.

Worked example At the root, sunny beat overcast by 0.153 to 0.147, a gap of 0.006 in Gini. The listings below refit a one-question tree 28 times, each time leaving out one day, and record the question it asks. Half the refits ask about sunny and half about overcast. Leaving out one sunny dontPlay day, or one rain Play day, is enough to swap the root, and every question below it is then chosen on a different set of days.

R

```r
first <- sapply(seq_len(nrow(w)), function(i) {
  t <- rpart(Decision ~ ., data = w[-i, ],
    method = "class",                 # day i out
    control = rpart.control(maxdepth = 1))
  labels(t, minlength = 0)[2]         # root question
})
table(first)
# Weather=rain,sunny      Weather=sunny
#                 14                 14
```

Python

```python
first = []                    # the root question
for i in range(len(w)):       # with day i left out
    t = tree.DecisionTreeClassifier(
        max_depth=1, random_state=0).fit(
        X4.drop(index=i), w["Decision"].drop(index=i))
    first.append(X4.columns[t.tree_.feature[0]])
pd.Series(first).value_counts()
# Weather_sunny       14
# Weather_overcast    14
```

R writes the overcast root as `Weather=rain,sunny`, the same question asked the other way round.

The rules move more than the labels A changed root does not mean changed predictions. The listings below refit the running tree, 5-NN on scaled predictors and LDA 28 times each, leaving out one day, and count how many of the other 27 days change label against the fit on all 28. Table 7 gives the averages. LDA's labels are the steadiest. scikit-learn's tree moves about as many labels as 5-NN, and `rpart`'s moves more, because the two libraries grow different trees on some of the 27-day samples. In both, the rules a reader would quote change in half the refits while most labels stay put. A fitted tree is best read as one of several near-equal descriptions of the data.

Table 7. Labels that change on the other 27 days when one day is left out, averaged over the 28 refits

	R	Python
Tree, the running controls	2.36	0.89
5-NN, scaled predictors	0.93	0.93
LDA	0.46	0.50

R

```r
library(MASS)     # lda
library(class)    # knn
X <- model.matrix(~ Weather + Temp + Humidity +
                  Wind, w)[, -1]
moved <- function(fit_predict) { # per refit
  full <- fit_predict(1:28)
  mean(sapply(1:28, function(i) {   # day i out
    p <- fit_predict(setdiff(1:28, i))
    sum(p[-i] != full[-i])          # of 27 days
  }))
}
ctl <- rpart.control(cp = 0, maxdepth = 3,
                     minsplit = 6, minbucket = 3)
round(c(
  tree = moved(function(r) predict(rpart(
    Decision ~ ., w[r, ], method = "class",
    control = ctl), w, type = "class")),
  knn = moved(function(r) {
    m <- colMeans(X[r, ]); s <- apply(X[r, ], 2, sd)
    Z <- scale(X, m, s)          # train scaling
    knn(Z[r, ], Z, w$Decision[r], k = 5)
  }),
  lda = moved(function(r) predict(lda(X[r, ],
    w$Decision[r]), X)$class)), 2)
# tree  knn  lda
# 2.36 0.93 0.46
```

Python

```python
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

Xd = pd.get_dummies(w.drop(columns="Decision"),
    dtype=float, drop_first=True).to_numpy()
yd = w["Decision"].astype(str).to_numpy()

def moved(make):     # labels that change, per refit
    full = make().fit(Xd, yd).predict(Xd)
    out = []
    for i in range(len(yd)):           # day i out
        keep = np.arange(len(yd)) != i
        p = make().fit(Xd[keep], yd[keep]).predict(Xd)
        out.append((p != full)[keep].sum())
    return round(np.mean(out), 2)      # of 27 days

moved(lambda: tree.DecisionTreeClassifier(
    max_depth=3, min_samples_split=6,
    min_samples_leaf=3, random_state=0))   # 0.89
moved(lambda: make_pipeline(StandardScaler(),
    KNeighborsClassifier(5)))              # 0.93
moved(LinearDiscriminantAnalysis)          # 0.5
```

13 Trees in R and in Python

What it is for The two libraries differ more in their tree defaults than anywhere else in this course, so a tree in one language is not the same tree in the other unless you set the stopping rules yourself.

13.1 Matched controls

Worked example Fit the same 28 cases on `Temp` and `Wind` alone, the tree of Section 9, in both languages with Gini and explicit depth and leaf-size controls. The Python fit reproduces the four-leaf partition in Table 8. R keeps three leaves, because even at `cp = 0` `rpart` drops a split that lowers Gini but not the training errors. With `cp = -1` it keeps such splits and matches scikit-learn's leaf counts in Table 5.

Table 8. Python leaf counts and predictions for the two-question tree

Leaf	n	Play	dontPlay	Predicts
Temp<14.5, *Wind*<88	3	3	0	Play
Temp<14.5, *Wind*≥88	3	2	1	Play
Temp≥14.5, *Wind*<91.5	13	5	8	dontPlay
Temp≥14.5, *Wind*≥91.5	9	6	3	Play

R

```
fit <- rpart(Decision ~ Temp + Wind, data = w,
  method = "class",
  parms = list(split = "gini"),
  control = rpart.control(cp = 0, maxdepth = 2,
    minsplit = 6, minbucket = 3))
fit
# n= 28
# 1) root          28 12 Play
#   2) Temp>=14.5  22 11 dontPlay
#     4) Wind<91.5 13  5 dontPlay *
#     5) Wind>=91.5 9  3 Play     *
#   3) Temp<14.5    6  1 Play     *
```

Python

```
from sklearn import tree
X = w[["Temp", "Wind"]]
fit = tree.DecisionTreeClassifier(
    criterion="gini", max_depth=2,
    min_samples_split=6, min_samples_leaf=3,
    ccp_alpha=0.0, random_state=15
).fit(X, w["Decision"])
print(fit.classes_)          # code-point order
# ['Play' 'dontPlay']
names = ["Temp", "Wind"]
print(tree.export_text(fit, feature_names=names,
                       show_weights=True))
# |--- Temp <= 14.50
# |   |--- Wind <= 88.00
# |   |   |--- weights: [3.00, 0.00] class: Play
# |   |--- Wind >  88.00
# |   |   |--- weights: [2.00, 1.00] class: Play
# |--- Temp >  14.50
# |   |--- Wind <= 91.50
# |   |   |--- weights: [5.00, 8.00] class: dontPlay
# |   |--- Wind >  91.50
# |   |   |--- weights: [6.00, 3.00] class: Play
```

The warm branch has the same two leaves in both fits. On the cold branch, Python splits the six cold days into groups of three with counts $(3, 0)$ and $(2, 1)$ for (Play, dontPlay). That split lowers Gini but leaves one day misclassified, exactly as before it, so R removes it. The Python seed is required too. At the six-case cold node two questions remove the same mixing, and scikit-learn tries the predictors in a random order, so `random_state = 15` fixes which one wins.

13.2 The defaults

Table 9 lines up the two libraries' defaults. `rpart` stops early by default. On the 28 rows it returns three leaves with ten training errors. `DecisionTreeClassifier` leaves the size of the tree to you and grows until every leaf is pure, 12 leaves with no training error. Comparing those fits without reporting the controls blames the programming language for different modelling decisions.

R

```
small <- rpart(Decision ~ Temp + Wind, data = w)
small     # defaults: cp = 0.01, minsplit = 20
# 1) root         28 12 Play
#   2) Temp>=17   20 10 dontPlay
#     4) Wind<92.5 12 5 dontPlay *
#     5) Wind>=92.5 8 3 Play     *
#   3) Temp<17     8  2 Play     *
```

Python

```
big = tree.DecisionTreeClassifier(random_state=15)
big.fit(X, w["Decision"])
(int(big.get_n_leaves()), int(big.get_depth()),
 float(big.score(X, w["Decision"])))
# (12, 7, 1.0), so 12 leaves, depth 7, no error
leaf = big.tree_.children_left == -1
sorted(big.tree_.n_node_samples[leaf].tolist())
# [1, 1, 1, 1, 2, 2, 2, 2, 3, 4, 4, 5] days a leaf
```

Table 9. Default settings compared between rpart and DecisionTreeClassifier

	`rpart`	`DecisionTreeClassifier`
Growth control	`cp = 0.01`, `minsplit = 20`, `minbucket = 7`, `maxdepth = 30`	`ccp_alpha = 0.0`, `min_samples_split = 2`, `min_samples_leaf = 1`, `max_depth = None`
Consequence	**prunes as it grows**: discards any split not worth 1% of the root's lack of fit, and does not split a node holding fewer than 20 cases	**grows to purity**: keeps splitting until every leaf is pure or cannot be split further
Categorical predictors	accepted directly; searches subsets of the levels, as in the Weather example	**not accepted**; encode first, and one-hot encoding restricts the search to one-level-versus-rest tests
Missing values	**surrogate splits** (`usesurrogate = 2`): a correlated backup test routes the case, and `summary()` names the surrogate and its agreement	no surrogates. Older versions raise `ValueError` on `NaN`; current ones learn a single default direction per split
Criterion	`parms = list(split = "gini")` (default) or `"information"`	`criterion="gini"` (default) or `"entropy"`
Regression	`method = "anova"`	`DecisionTreeRegressor`

13.3 Categorical predictors

`rpart` takes `Weather` as a factor and searches groupings of its levels, as Section 4 describes. scikit-learn needs numbers, so `Weather` enters as indicator columns, one per level.

R

```r
t1 <- rpart(Decision ~ Weather, data = w,
  method = "class",
  control = rpart.control(cp = 0, minsplit = 2,
                          maxdepth = 1))
t1        # Weather = sunny versus the rest
```

Python

```python
Xw = pd.get_dummies(w[["Weather"]])  # all 3 levels
t1 = tree.DecisionTreeClassifier(max_depth=1)
t1.fit(Xw, w["Decision"])
names = list(Xw.columns)
print(tree.export_text(t1, feature_names=names))
# |--- Weather_sunny <= 0.50
# |   |--- class: Play
# |--- Weather_sunny >  0.50
# |   |--- class: dontPlay
```

A one-level question on indicator columns asks "is Weather $= L$?" for one level L, one level against the rest, and here it chooses sunny, the same question as `rpart`. With three levels every grouping is one level against the rest, so nothing is lost. With many levels a useful grouping needs several indicator splits, and the tree may stop before it assembles them.

13.4 Missing values

`rpart` [4] routes a case with a missing predictor through a *surrogate*, another predictor whose question agrees as closely as possible with the primary one. scikit-learn has no surrogates. Since version 1.3 its trees accept missing values directly. At each candidate cut they try sending every case with a missing value to the left and then to the right, and keep the better score. Older versions need the values imputed, with an imputer fitted on the training rows only.

Differences to know

- `rpart` stops early by default (`cp = 0.01`, `minsplit = 20`, `minbucket = 7`). `DecisionTreeClassifier` grows until every leaf is pure.

- `rpart` searches groupings of a factor's levels. scikit-learn needs indicator columns, which cut off one level at a time.
- `rpart` routes missing values with surrogates. scikit-learn has none.
- scikit-learn breaks ties between questions at random, so set `random_state`.

14 Common misconceptions

1. *"The tree found the most important variables."* It found the variables that were best *first*, greedily. A variable that never appears may be as informative as one that does, if a related variable was used before it.
2. *"Zero training error means the tree is good."* When no two cases share their predictor values with different labels, a large enough tree reaches zero training error by construction. On `Glass` the training score reached 0.99 while the outer folds stayed near 0.61.
3. *"The impurity measure tells you when to stop."* It never does. Training impurity can only fall, so it always favours another split.
4. *"A pure child means a good question."* Only if it takes enough of the data with it. The overcast question gives a pure child and still loses to the sunny question on Gini, because the pure child holds 8 of 28 days.
5. *"I should standardize the predictors first."* Not for a tree. Only the order of a numeric predictor's values matters.
6. *"A tree is less accurate than an SVM."* On `Glass` the tuned tree's gap to the SVM is $+0.015$, with a 95% interval from -0.30 to 0.33. Five folds cannot separate them.
7. *"`rpart` and scikit-learn disagree, so one of them is wrong."* Their defaults encode different intentions. Set the stopping rules yourself.
8. *"The root question is the most important fact about the data."* Leaving out one of 28 days swaps the root between sunny and overcast. A question that wins by 0.006 could as easily have lost.

15 Summary

A decision tree predicts with a short list of yes-or-no questions, which makes it a model you can read. It is grown by recursive partitioning. At each node it lists the questions each predictor allows, scores them, keeps the best, splits, and repeats inside each side, greedily and top-down.

A question is scored by the mixing it removes. Gini impurity is the chance that two random cases from a box disagree, and a question is worth the drop from the parent's Gini to the weighted average of its children's. That weighting is why a small pure child is worth less than it looks. Entropy scores the same way and charges lopsided boxes more, which on the weather data moves the root from sunny to overcast. A new case is predicted by following its answers to one leaf, whose counts are its class shares.

A tree on two numeric predictors is a set of axis-aligned rectangles, so a diagonal boundary becomes a staircase. Training accuracy only rises as the tree grows, and an unstopped tree memorizes its rows. The depth is a setting chosen by cross-validation, and on `Glass` five folds cannot separate the tuned tree from kNN and the SVM. Leaving out one day changes the root half the time and few of the predicted labels. Coming next, trees for numeric targets are grown and pruned back to the simplest tree the evidence supports.

References

[1] L. Breiman, J. Friedman, R. Olshen, and C. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. "Scikit-learn: machine learning in Python". In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830.

[3] J. R. Quinlan. "Induction of decision trees". In: *Machine Learning* 1.1 (1986), pp. 81–106.

[4] T. Therneau and E. Atkinson. *An Introduction to Recursive Partitioning Using the RPART Routines*. Tech. rep. Distributed as the `rpart` package vignette. Mayo Foundation.